

Wallet Guide

From a seed to an authorised payment

m@rek.onl

Abstract

This volume of *The Zcash Arboretum* constructs the wallet side of a modern shielded payment, from reproducible secret keys to a transaction whose exact payments its signer has approved. It assumes the *Math Guide*'s encodings, the *Crypto Guide*'s key derivation and wallet permutations, the *Consensus Guide*'s branch-local acceptance rules, and the *Ironwood Guide*'s complete payment construction. The endpoint is an algorithm that turns a selected set of spendable notes and a payment intent into a verifiable transaction, while preserving the distinctions between authority, privacy and chain state. The exposition is non-normative and does not treat wallet policy as consensus.

Contents

1	From seed to account	3
1.1	Master and hardened child derivation	3
1.2	Account paths and scopes	4
1.3	Key identifiers are not authority	4
2	Addresses and receiver selection	4
2.1	Indexed diversifiers	4
2.2	A canonical Unified Address	5
2.3	Selecting a receiver and a pool	6
3	Fees, inputs, and change	6
3.1	What makes a candidate spendable	6
3.2	The conventional fee	6
3.3	Padding and the fixed-point problem	7
3.4	The shared payment	8
4	Constructing and authorising a transaction	8
4.1	Construction order	8
4.2	Separated roles and capabilities	9
4.3	What the signer must check	9
4.4	Submission is an uncertain operation	10
5	Tracking payment state and recovery	10
5.1	Pending, mined, conflicted and expired	10
5.2	Reorganisation and witness freshness	11
5.3	What a seed backup can recover	11
5.4	Acceptance invariants	11

1 From seed to account

The payment protocol needs a spending key. A wallet additionally needs a recoverable organisation of many keys: distinct accounts, different networks and a sequence of receiving addresses. A deterministic key tree makes those objects reproducible from a single secret seed without making the seed public.

The underlying key and note types come from the *Ironwood Guide*, “Keys and viewing capabilities”, and its complete construction in “One payment, end to end”. This volume’s byte hashes and fixed-width encodings use the *Crypto Guide*, “Hash functions and key derivation”, and the *Math Guide*, “Language, integers, and encodings”. Those constructions precede the derivations below.

A *seed* here is the exact secret byte string accepted by ZIP 32, not the user interface’s written representation of it. *Entropy* quantifies a random choice’s unpredictability. For guessing resistance, k bits of *min-entropy* means no single seed has probability greater than 2^{-k} ; a uniform 32-byte seed meets $k = 256$. The probability language is constructed in the *Math Guide*, “Probability and computation”. ZIP 32 requires a length between 32 and 252 bytes and recommends at least 256 bits of entropy. Length alone does not establish entropy: thirty-two predictable bytes are not a secure seed. If an application represents the seed through words and a passphrase, its exact conversion and normalisation rules are part of recovery. A different passphrase can derive an entirely different wallet without any parsing error.

The source baseline is ZIP 32 at zips commit 69610984, orchard 0.15.0 src/zip32.rs, and librustzcash commit e30517e4. This is a construction pin, not a statement that every wallet implements every newer rule set described below.

1.1 Master and hardened child derivation

A *master* key begins the derivation tree. A *child* is obtained by applying a specified function to a parent and index. *Hardened* means this function consumes the parent’s spending secret, not merely its public or viewing data.

An extended Orchard spending key is (sk, c) : a 32-byte spending key and a 32-byte *chain code*, a separate derivation secret. From seed S , compute

$$I = \text{BLAKE2b}_{512}(\text{ZcashIP32Orchard}, S), \quad sk_m = I[0 : 32], \quad c_m = I[32 : 64].$$

The personalisation is exactly 16 bytes. The chain code is not an additional amount of independently sampled entropy; both halves derive from the same seed.

For a child index i satisfying $2^{31} \leq i < 2^{32}$, set

$$I = \text{PRFexpand}_{c_{\text{par}}}(\text{0x81} \parallel sk_{\text{par}} \parallel \text{enc}_4(i)), \quad (sk_i, c_i) = (I[0 : 32], I[32 : 64]).$$

The expansion function is constructed in the *Ironwood Guide*, “Keys and viewing capabilities”. A non-hardened index $i < 2^{31}$ is rejected. The generic ZIP 32 mechanism also has optional tag extensions; ordinary Orchard derivation uses no additional tag or lead-byte payload, which preserves the displayed preimage.

The notation j' means the encoded index $j + 2^{31}$, for $0 \leq j < 2^{31}$. Hardened derivation consumes the parent spending secret, so an exported full viewing key cannot derive sibling or descendant account spending keys. It can still generate diversified receiving addresses within the account as described below.

A derived 32-byte spending key must pass the protocol’s validity tests: a nonzero authorising scalar and valid nonzero incoming-viewing derivations in both external and internal scopes. A failure is not permission to silently invent a different derivation at the same path. The API must return failure or follow a separately specified retry/allocation rule that recovery reproduces.

1.2 Account paths and scopes

The standard account path is

$$m_{\text{Orchard}}/32'/\text{coin_type}'/\text{account}'.$$

For Mainnet, the coin type is 133; test networks use 1. The account index starts at zero and is conventionally allocated sequentially. Account a is an independent bucket of spending authority, not an output position or a diversifier index.

For Mainnet account 9, the three child indices are

$$0x80000020, \quad 0x80000085, \quad 0x80000009.$$

Each child applies the displayed derivation to the preceding extended key. The resulting sk feeds the complete protocol key hierarchy in the *Ironwood Guide*, “Keys and viewing capabilities”: spend scalar, full viewing key, incoming key, diversifier key and outgoing key.

Internal change scope is not an extra child-path level. It uses the protocol’s 0x83 derivation of $r_{\text{ivk_int}}$ while retaining the spend-authorising key and nullifier key. The resulting incoming and outgoing viewing keys differ. A wallet should record which scope detected a note, since it determines the receiving-key derivation and recovery capabilities for later use.

The *wallet birthday* is the earliest height from which recovery must scan for possible receipts. An account’s derivation path, network and wallet birthday are not secret substitutes for its keys, but can be sensitive metadata. Export only the capability the consumer needs. A view-only wallet should not receive an extended spending key merely to avoid defining a smaller export.

1.3 Key identifiers are not authority

For raw full-viewing-key encoding $F = \text{ak}^* \parallel \text{enc}_{32}(\text{nk}) \parallel \text{enc}_{32}(r_{\text{ivk}})$, the Orchard fingerprint is

$$\text{BLAKE2b}_{256}(\text{Zcash0rchardFVFP}, F).$$

Its first four bytes form a lookup tag. A short tag can collide and must not be accepted as proof of key identity or as a substitute for comparing the full key. Even a full fingerprint is only a digest of data, not proof that the caller controls the corresponding spending scalar.

The extended-key binary representation stores depth, parent tag, child index, chain code and spending key. Its widths are $1 + 4 + 4 + 32 + 32 = 73$ bytes. The root uses depth zero, zero parent tag and zero child index. The metadata supports navigation and checks; changing it does not perform a new cryptographic child derivation.

2 Addresses and receiver selection

An account can receive many payments without introducing a new spending key for each address. A *receiver* is the protocol-specific payment target. A *Unified Address* is a checksummed container offering one or more receiver types. The container is not itself a shielded note and is not stored as a recipient field in an ordinary Action.

2.1 Indexed diversifiers

The exact FF1 algorithm, its tweak parameter and its AES primitive are constructed in the *Crypto Guide*, “Wallet permutations and address encodings”, with the byte-field arithmetic in the *Math Guide*, “Byte fields for AES”. Given an account’s diversifier key dk and an index $0 \leq j < 2^{88}$, derive

$$d_j = \text{FF1-AES256.Encrypt}(\text{dk}, \epsilon, \text{LE}_{88}(j)).$$

The tweak ϵ is empty. The message is an 88-bit string with the specified little-endian bit ordering.

From d_j , compute g_{d_j} and $pk_{d_j} = [ivk]g_{d_j}$ by the *Ironwood Guide*, “Keys and viewing capabilities”. Every Orchard diversifier index yields a valid diversified base. The default index is zero. Sequential allocation permits a wallet to reconstruct an issued sequence, while the keyed permutation conceals its visible arithmetic structure under its security assumption.

Invertibility has a precise limit. Anyone knowing dk can invert FF1 on *every* 11-byte diversifier, not only ones belonging to the wallet. To recognise ownership of an entire receiver, recover its index and additionally recompute the transmission key with the account’s ivk and compare it to the supplied pk_d . FF1 inversion alone is not an ownership test.

Different addresses from one incoming key do not require one decryption attempt per address. Trial decryption uses the incoming scalar, then recovers the diversifier from the plaintext. More accounts and scopes, by contrast, can require additional trial-decryption work.

2.2 A canonical Unified Address

For the widely used revision-0 format, receiver typecodes are 0x00 for a transparent public-key hash, 0x01 for a transparent script hash, 0x02 for Sapling and 0x03 for Orchard. A container has at most one item of each type, cannot include both transparent alternatives, and must contain at least one shielded item. The transparent hash receivers are 20 bytes; the two shielded raw receivers are 43 bytes.

Serialize items in ascending typecode order:

$$M_0 = \left\| \left(\text{CompactSize}(\text{type}_i) \parallel \text{CompactSize}(|R_i|) \parallel R_i \right) \right\|_i$$

The CompactSize encoding is constructed in the *Consensus Guide*, “Blocks and branch-local validation”. Both typecode and length are bounded by 0x2000000, in addition to the total-size bound. No repeated item, noncanonical integer or trailing unparsed byte is permitted.

Let h be the human-readable prefix in ASCII, padded with zero bytes to 16 bytes. For revision-0 Mainnet Unified Addresses it is the byte 0x75, representing u , followed by fifteen zero bytes. Encode

$$\text{Bech32m}_u(\text{F4Jumble}(M_0 \parallel h)),$$

using ZIP 316’s extended length allowance rather than the ordinary short Bech32m limit. F4Jumble and Bech32m are constructed in the *Crypto Guide*, “Wallet permutations and address encodings”. Jumbling is a public invertible permutation, not encryption of the receiver. Anyone can decode the address.

The smallest Orchard-only revision-0 item begins with 03 2b and contains its 43 receiver bytes. Its raw size is 45 bytes, and its padded jumble input is 61 bytes. For one transparent hash receiver plus Sapling and Orchard, the three item lengths are 22, 45, 45, giving 128 bytes after the prefix padding. These lengths determine the exact permutation split; dropping the network padding changes the encoded address.

Decode in reverse: check the recognised network prefix and Bech32m checksum; invert F4Jumble within the accepted length range; verify and remove the exact 16-byte prefix padding; parse all canonical items; validate item ordering, uniqueness, allowed combinations and supported receiver encodings. A valid checksum detects many copying errors, but does not authenticate that the address belongs to the intended human.

Later ZIP 316 revisions add distinct prefix and item semantics. A consumer selects the rules from the encoded revision, not from a guess about its contents. Unknown items are handled by that revision’s ignore-or-reject rules, including any must-understand category. The revision-0 construction above must not silently accept a newer container under the wrong parser.

2.3 Selecting a receiver and a pool

For the basic receiver list, preference is Orchard, then Sapling, then the supported transparent alternative. A sender chooses the highest priority receiver it supports from those actually present. Serialisation order is ascending typecode; preference order is different. Sorting the container and choosing its first element would therefore select the wrong receiver.

Receiver support is not just a decoding capability. A wallet must be able to construct a valid transaction for the selected network, target height and pool rules. In the specified Ironwood profile, the existing Orchard receiver type also receives Ironwood-protocol outputs. No extra address typecode is required merely to distinguish those pools. The transaction component and applicable rules select the pool; the note's receiver bytes do not.

A wallet may reject a payment it cannot safely perform. It should not silently replace a supported preferred shielded receiver with a transparent receiver to make its current input-selection code succeed. Such a fallback changes the public information disclosed by the payment.

A payment request must be resolved to exact recipient bytes and an integer zatoshi amount before transaction construction. Decimal parsing must be exact: floating-point rounding is not a valid way to interpret a requested monetary amount. A memo and human label remain separate metadata; neither overrides the receiver selected from the address.

3 Fees, inputs, and change

The wallet starts with an intent and a set of candidate notes. It must choose inputs and a transaction shape together because the shape affects the fee, while the fee affects whether change exists.

3.1 What makes a candidate spendable

For each candidate note, the wallet needs its amount, seed and receiving scope; pool and creation identifier; commitment position; nullifier when available; branch-linked mined location; and a path to a selected eligible anchor. It must also know that the note is not already spent on that branch and is not reserved by another pending transaction the wallet is still pursuing.

A displayed balance is not necessarily selectable value. It may include unconfirmed receipts, notes without available witnesses, notes below a chosen confirmation threshold, or notes in a pool whose transition rules restrict the intended transaction. A view-only wallet can know value that it cannot authorise.

Let the observed selected tip have height H and the next target height be $h = H + 1$. A wallet choosing an anchor k blocks behind that target uses height $a = h - k$, provided the branch contains an eligible root there. A note created later than a is not in that anchored tree. The numerical choice of k is wallet policy; consensus eligibility of a historical root does not impose that policy. All inputs of one bundle must be witnessed against its same anchor.

Different pools have separate roots and tree histories. A transaction using two pools can need two independently constructed witness sets. A common height is convenient but does not make their root bytes equal.

3.2 The conventional fee

ZIP 317's basic conventional fee is

$$f_{\text{conv}} = 5000 \max(2, L) \text{ zatoshis,}$$

where L is the total number of logical actions. For the component formats used here,

$$\begin{aligned} L_T &= \max\left(\left\lceil \frac{s_{\text{in}}}{150} \right\rceil, \left\lceil \frac{s_{\text{out}}}{34} \right\rceil\right), \\ L_{\text{Sprout}} &= 2n_{\text{JoinSplit}}, \\ L_{\text{Sapling}} &= \max(n_{\text{spend}}, n_{\text{output}}), \\ L_{\text{Orchard}} &= n_{\text{Actions}}. \end{aligned}$$

The transparent quantities $s_{\text{in}}, s_{\text{out}}$ are the serialized input and output field sizes in bytes, not simply counts. The Ironwood fee revision adds its Action count as another contribution. Only apply additional fee-revision terms when their specified format is in use; unrelated memo-bundle proposals are not part of this construction.

The total is the sum of the relevant contributions. A modern Action pairs one input side and one output side, so it counts once, not twice. Padding is visible in the Action count and contributes to the fee even when its private value is zero.

The conventional fee is policy, not a consensus minimum. Ordinary consensus still requires a valid nonnegative fee computed from the public balance equation in the *Consensus Guide*, “Ledger state and transaction acceptance”. Paying that nonnegative amount does not imply a node will relay or a miner will include the transaction. Following a common fee convention also avoids making a wallet’s unusual fee choice an unnecessary fingerprint.

3.3 Padding and the fixed-point problem

Here a *fixed point* means a proposed shape, fee and change amount that agree when each is recomputed from the others. For an ordinary nonempty Orchard-protocol bundle with s real input notes and o intended output notes, a common builder padding policy uses

$$m = \max(2, s, o)$$

Actions. Add zero-valued dummy sides until both lists have m entries. The protocol permits the zero-value membership gate; the minimum of two in this policy is not a general consensus minimum of two Actions.

If payment totals P and selected inputs total V , the prospective change is

$$C = V - P - f(m).$$

But adding a change output can increase o , hence m , hence the fee. A correct selection algorithm must evaluate the actual resulting shape, not calculate a fee once before deciding the outputs.

A *dust policy* specifies how to treat a residual too small for the wallet’s chosen minimum economical change output. The threshold and treatment are wallet choices, not new consensus amount limits.

A simple terminating algorithm for one supported pool is:

1. Start with a finite ordered candidate list satisfying the anchor and spendability conditions. State the ordering policy, such as oldest first; it is not a consensus rule.
2. Add a candidate input and compute the shape with no change. If its input total is insufficient for payments and that shape’s fee, continue.
3. Evaluate the shape including change and its resulting fee. If $C = V - P - f_{\text{with change}}$ is a permitted positive change amount, retain that shape.
4. If there is exact equality for the no-change shape, retain no change. For a small residual, apply the explicitly chosen dust policy: add it to the fee, reject the proposal, or continue selecting inputs.

5. Recompute all totals, padding and fee from the final proposed inputs and outputs. Return failure when candidates are exhausted.

The input list grows on every retry, so the algorithm terminates. It is not guaranteed to minimise the fee or the number of linked notes. More sophisticated selection can improve those objectives without changing the correctness conditions.

A small note may be unhelpful if adding it raises the fee by more than its value. The two-action grace window means this is not equivalent to rejecting every note below 5000 zatoshis: some such notes add no marginal Action cost in a given shape. Dust decisions depend on the whole proposal.

3.4 The shared payment

For the 80,000 input, 50,000 payment and one prospective change output, the final counts are $s = 1, o = 2, m = 2$. Thus

$$f = 5000 \max(2, 2) = 10,000, \quad C = 80,000 - 50,000 - 10,000 = 20,000.$$

The two-Action shape is stable after including change. A 10,000 fee is not an additional hidden note; it is the positive public net withdrawal from the pool, subsequently available to the block's coinbase under its accounting rules.

The input-to-output pairing can use the real input with the payment and the dummy input with change. The corresponding private differences are 30,000 and $-20,000$, summing to the public balance. The builder may randomise pairing and Action order, but must preserve the resulting note-creation identifiers, ciphertext associations, public instances and signature order.

4 Constructing and authorising a transaction

A *proposal* is an approved choice of inputs, outputs, fee, pool routing and intended validity context. It is not yet a transaction proof or a signature. Freezing this semantic object makes the remaining dependencies explicit.

4.1 Construction order

The *Ironwood Guide*, “One payment, end to end”, supplies the exact cryptographic equations. A wallet executes them in the following order:

1. Resolve and validate recipient receivers, zatoshi amounts and memo bytes. Derive change receivers in the intended account and scope.
2. Select target rules, transaction version and branch identifier. Select input notes, pool-specific anchors and witness paths; compute fee, change and padding to a stable final shape.
3. Reconstruct each old note and nullifier. Generate all required dummy old notes and their fresh keys. Choose the final Action order and old/new pairing.
4. Set each new note's ρ to its paired old nullifier. Sample fresh note seeds, derive esk and ψ , then the correct note-version rcm . Compute new commitments.
5. Sample spend-key randomisers and value blinders. Form randomised keys and net commitments. Derive incoming ciphertexts and optional outgoing recovery ciphertexts using those public Action fields.
6. Assemble the exact effecting-data representation and public proof instances. Generate the proof with the selected circuit and witnesses.

7. Recompute the complete signature digest. Obtain spend signatures and the binding signature. Form authorising-data commitments.
8. Serialize canonically. Independently verify proof and signatures, recompute the fee and check the intended outputs before submission. Store the pending record and reserved inputs atomically.

Generating the proof before the signatures is a convenient implementation order, not a cryptographic dependency in every supported format. Both depend on the final effecting data. A changed recipient, amount, ciphertext, flag or public balance can invalidate the proof, signatures or both; a builder must know which dependency it changed.

For the specified version-6 digest, re-anchoring does not change the effecting-data identifier, but it does change proof instances and authorising data. This limited property must not be generalised to a changed output or to version 5 anchors.

4.2 Separated roles and capabilities

A Partially Constructed Zcash Transaction, or *PCZT*, is a container for coordinating construction before all proofs and signatures exist. Its purpose is to separate capabilities, not to change consensus validity. The on-chain product remains an ordinary transaction.

A constructor selects the intended structure and randomness; a prover needs private witnesses; a signer controls spending authority; an extractor assembles and validates the final serialised transaction. One program may perform several roles. A hardware signer or remote prover can perform just one.

Role	Required information and resulting boundary
Planner	Payments, candidate values, pool rules, anchors and fee policy; can propose a transaction but cannot necessarily authorise it.
Prover	Notes, paths, viewing-key components, randomisers and blinders; learns the supplied witness even though chain observers do not.
Spend signer	Spending authority and enough authenticated construction data to approve the entire signed effect. It need not generate Halo 2 proofs.
Finaliser	Complete effecting and authorising data; verifies and extracts the transaction. Missing approval is not repaired by serialisation.

The Action witness does not contain ask. A prover knowing the required note, full viewing key and randomiser can construct the relation without spending authority. A remote prover therefore need not receive ask, but it does receive sensitive note data unless an additional privacy mechanism is designed. Proof zero knowledge protects the *verifier* from that witness, not the machine computing it.

For a spend randomiser α , the signer uses $rsk = ask + \alpha$ and returns a signature, not rsk . Sending rsk to a coordinator who knows α reveals $ask = rsk - \alpha$. Algebraic key randomisation is not a reason to export the randomised secret to an untrusted constructor.

The binding signature authorises the aggregate commitment balance. Its secret is the sum of value blinders when the proposal balances. A multi-party construction must arrange that capability deliberately; calling it a separate role does not by itself define a secure multi-party blinder or signing protocol.

4.3 What the signer must check

An untrusted coordinator can supply internally consistent data describing the wrong payment. The signer must bind its decision to user intent:

1. Check the network, transaction version, consensus branch, expiry and any lock-time semantics against its policy and the intended construction context.

2. Identify each claimed wallet input by its actual note data and key derivation. Recompute the nullifier and randomised public key. Do not trust an account label or short fingerprint as proof of ownership.
3. Independently derive every claimed change receiver and scope. Reconstruct its output note and commitment. A coordinator's label *change* does not make an output belong to the signer.
4. Check external recipients, amounts and memos against the approved payment request. Recompute output commitments and, when delivery is promised, the ciphertext construction from the supplied output seed.
5. Compute integer input and output totals, public pool balances and actual fee. Check padding and resource bounds, and authenticate previous transparent output data if such inputs are included.
6. Recompute the precise signature digest from the verified data. Sign only that digest using the correct Action randomised key.

An implementation can replace some local recomputation with a separately justified proof or authenticated attestation. It cannot replace it with unverified coordinator assertions without changing the trust model.

Signer intent checks and node validity checks overlap but are not identical. A node need not know that Bob was the intended payee. A signer need not download and validate the whole blockchain merely to approve Bob's amount, but should know what state assertions it is trusting. A hardware display that shows only an untrusted host-provided amount does not establish a verified authorisation boundary.

4.4 Submission is an uncertain operation

Persist the transaction and input reservations before handing it to a network service. A timeout after submission is ambiguous: the service may have received and relayed the transaction even though the reply was lost. Retrying the same transaction can be safe; constructing a conflicting replacement without recording that ambiguity can cause inconsistent wallet state.

A successful submission response means the service accepted the request under its own behaviour. It does not prove relay to miners, inclusion, or finality. Track the transaction by its exact identity and later branch-linked mined location. Fees are paid through mined transaction effects, not by the act of requesting broadcast.

5 Tracking payment state and recovery

A wallet maintains an interpretation of the selected chain plus its own pending intentions. These two sources of state must remain distinct.

5.1 Pending, mined, conflicted and expired

A useful local lifecycle distinguishes:

- *Constructed*: complete transaction retained, perhaps not yet submitted.
- *Submitted or pending*: the wallet is pursuing inclusion; selected inputs remain reserved.
- *Mined on the selected branch*: inclusion is recorded by block hash, height and transaction position.
- *Conflicted*: another selected-chain transaction consumes a reserved input or nullifier.
- *Expired*: the active expiry rules no longer permit future inclusion at the relevant target height.

These labels are local interpretations, not extra consensus fields. A transaction can move from mined back to pending after a reorganisation. It can subsequently conflict or expire. A dropped network connection does not prove that a pending transaction was never received elsewhere.

Input reservations prevent accidental concurrent construction using the same notes. They do not change consensus spendability. Release them only under a recorded cancellation, expiry, conflict or replacement policy; restore the appropriate reservations after rollback.

5.2 Reorganisation and witness freshness

The *Consensus Guide*, “Reorganisation and confirmation confidence”, constructs the branch transition. The wallet mirrors its relevant effects. A note received only on a removed branch is no longer a mined input. A spend observed only there no longer proves that a note is spent on the replacement branch. A path must be rebuilt for the selected anchor; a path to a removed root does not become valid by relabelling its height.

Store block hashes with mined transaction records and note positions. After finding the common ancestor, roll back the chain-derived records together: received notes, spend observations, tree checkpoints and transaction confirmations. Preserve separately the user’s payment intent and retained signed transaction. Then evaluate whether that transaction can still be pursued.

A wallet can choose not to spend very recent change, or can impose a longer delay on external receipts than internal transfers. Such choices trade availability for its chosen risk tolerance. The choice should be described as policy, not as a cryptographic guarantee attached to the word *confirmed*.

5.3 What a seed backup can recover

A seed with the correct network and account derivation conventions recovers spending and viewing capabilities. Given complete authentic chain data, the wallet can rediscover decryptable incoming notes and derive their nullifiers to distinguish unspent value. It need not know every previously issued diversifier index before trial-decrypting: the incoming scalar is shared across that indexed address family.

Account discovery is a different problem. An implementation that stops after a fixed number of unused accounts assumes that later accounts were not used. That assumption can miss funds if the user previously skipped far ahead. Record account allocation or document the finite discovery policy rather than claiming that one arbitrary gap limit recovers every possible account.

The wallet birthday is an earliest scan height justified by creation or backup history. Setting it too late can omit received notes permanently from that scan. An earlier birthday costs extra work but does not authenticate the server supplying that history.

Seed-only recovery does not reconstruct arbitrary local labels, invoice associations, contact identities or all outgoing memos. Outgoing recovery requires the sender to have used the corresponding outgoing viewing key and the necessary full ciphertext data to remain available. Pending transactions not present in the recovered chain may need a separate backup if their exact signed bytes or intent matter.

5.4 Acceptance invariants

At the end of construction and throughout recovery, the wallet should maintain three invariants:

1. Every selectable note has coherent pool, branch, note data, position, unspent status and witness availability.
2. Every authorisation signs the exact effecting data approved by its signer, with amounts and change independently checked.
3. Every persisted chain-derived update can be associated with a validated or explicitly trusted branch point and rolled back without corrupting unrelated local intent.

These invariants are a constructive endpoint: they specify what an implementation must preserve while choosing its own database and user interface. They do not prescribe a catalogue of RPC calls or schemas.

Evidence and checks. The key and address constructions are ZIP 32 and ZIP 316; the conventional fee is ZIP 317. The implementation references are `orchard/src/zip32.rs`, `zcash_address/src/kind/unified.rs`, `zcash_client_backend/src/fees`, and the transaction-construction interfaces in `zcash_client_backend/src/data_api/wallet.rs` at the stated source pins. Separated roles are represented by `pczt/src/roles`, but their security depends on the explicit checks above, not the container name. The companion arithmetic check covers the shared fee/change example, encoding widths, and published ZIP 32 master and hardened-child vectors. It is not a production wallet implementation or a proof that a particular hardware signer performs these checks.